LAPORAN PRAKTIKUM

PEMROGRAMAN MOBILE

Pertemuan 6: Tema, Routing dan Navigasi, serta Animasi

Nama	:	Muhamad Dika Ramadhan
NIM	:	20240801045
Program Studi	:	Teknik Informatika
Mata Kuliah	:	Pemrograman Mobile
Pertemuan	:	6

1. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Membuat tema terang/gelap dan warna tema yang dapat diganti pengguna menggunakan ThemeData dan ThemeMode.
2. Memakai nilai tema melalui Theme.of(context) sebagai pengganti warna dan gaya yang ditulis manual.
3. Menerapkan named routes, mengirim argumen antar halaman, dan menggunakan onGenerateRoute.
4. Membuat navigasi tab bawah dengan NavigationBar dan IndexedStack.
5. Membuat animasi implisit (AnimatedContainer, AnimatedSwitcher), transisi Hero, dan animasi eksplisit (AnimationController).

2. Landasan Teori

Tema

ThemeData menyimpan seluruh gaya aplikasi, mulai dari warna, tipografi, sampai gaya komponen. Pada Material 3, cukup dengan menentukan satu warna dasar (seed color), Flutter akan menghasilkan seluruh palet warna secara otomatis. MaterialApp menerima tiga properti utama, yaitu theme untuk tema terang, darkTheme untuk tema gelap, dan themeMode untuk memilih antara sistem, terang, atau gelap. Widget membaca tema dengan Theme.of(context) sehingga tampilannya menyesuaikan secara otomatis ketika tema berubah.

Routing dan Navigasi

Selain Navigator.push secara langsung, halaman dapat diberi nama (named route) sehingga navigasi lebih rapi dan terpusat. Method yang sering dipakai ditunjukkan pada tabel berikut.

Perintah	Fungsi
pushNamed(ctx, '/detail', arguments: x)	Membuka halaman bernama sambil membawa argumen.
pop(ctx)	Menutup halaman saat ini dan kembali ke halaman sebelumnya.
pushReplacementNamed(...)	Mengganti halaman saat ini sehingga pengguna tidak dapat kembali, misalnya dari login ke beranda.
pushNamedAndRemoveUntil(...)	Membuka halaman baru sambil menghapus riwayat halaman sebelumnya.

Animasi

Flutter menyediakan tiga pendekatan animasi. Animasi implisit (AnimatedContainer, AnimatedOpacity, AnimatedSwitcher) cukup dijalankan dengan mengubah nilai melalui setState, kemudian Flutter menganimasikan perubahannya. Transisi Hero membuat sebuah elemen tampak berpindah mulus antar dua halaman selama tag-nya sama. Animasi eksplisit (AnimationController bersama RotationTransition dan sejenisnya) dipakai ketika dibutuhkan kontrol penuh, misalnya mengulang, menghentikan, atau menyinkronkan beberapa animasi.

3. Alat dan Bahan

- Flutter SDK, editor kode, dan emulator Android yang digunakan pada pertemuan sebelumnya.
- Proyek baru dengan nama praktikum_6 yang dibuat melalui perintah flutter create praktikum_6.
- Paket shared_preferences (dari Pertemuan 5) untuk menyimpan pengaturan tema pada latihan mandiri dan tugas.

4. Langkah dan Hasil Praktikum

Proyek praktikum_6 dikerjakan secara bertahap dalam empat bagian (A sampai D). Seluruh kode berada pada berkas lib/main.dart. Setiap bagian di bawah ini memuat potongan kode utama, penjelasan fungsi yang digunakan, dan hasil yang diperoleh.

Bagian A. Tema Terang/Gelap dan Warna Tema

Pada bagian ini dibuat dua ValueNotifier global untuk menyimpan mode tema dan warna dasar tema. ListenableBuilder membangun ulang MaterialApp setiap kali salah satu nilainya berubah, sehingga tema berganti seketika tanpa restart aplikasi.

```
final themeMode = ValueNotifier<ThemeMode>(ThemeMode.light);
final seedColor = ValueNotifier<Color>(Colors.indigo);

ThemeData buatTema(Color seed, Brightness brightness) {
  return ThemeData(
    useMaterial3: true,
    colorSchemeSeed: seed,
    brightness: brightness,
    appBarTheme: const AppBarTheme(centerTitle: true),
  );
}
```

```
// di dalam MyApp.build()
return ListenableBuilder(
  listenable: Listenable.merge([themeMode, seedColor]),
  builder: (context, _) {
    return MaterialApp(
      theme: buatTema(seedColor.value, Brightness.light),
      darkTheme: buatTema(seedColor.value, Brightness.dark),
      themeMode: themeMode.value,
      ...
    );
  },
);
```

Penjelasan fungsi yang digunakan:

Bagian Kode	Penjelasan
ValueNotifier<T>	Menyimpan sebuah nilai yang dapat diamati. Setiap kali value diubah, seluruh pendengarnya diberi tahu agar memperbarui tampilan.
themeMode	Notifier bertipe ThemeMode yang menyimpan pilihan mode tema (terang, gelap, atau mengikuti sistem). Nilai awalnya terang.
seedColor	Notifier bertipe Color yang menyimpan warna dasar tema. Nilai awalnya Colors.indigo.
pilihanWarna	Daftar konstan berisi empat warna (indigo, teal, deepOrange, pink) yang dapat dipilih pengguna.
buatTema()	Fungsi pembuat ThemeData. colorSchemeSeed membentuk seluruh palet Material 3 dari satu warna, brightness menentukan versi terang atau gelap, dan AppBarTheme(centerTitle: true) membuat judul AppBar berada di tengah.
main() dan runApp()	Titik awal program. runApp memasang MyApp sebagai widget akar aplikasi.
ListenableBuilder	Membangun ulang widget di dalam builder setiap kali objek listenable memberi sinyal perubahan.
Listenable.merge()	Menggabungkan beberapa notifier (di sini themeMode dan seedColor) menjadi satu objek yang diamati.
MaterialApp	Widget akar aplikasi Material. theme untuk tema terang, darkTheme untuk tema gelap, dan themeMode menentukan tema yang dipakai.
SwitchListTile	Baris berisi judul dan saklar. Properti value bernilai true saat mode gelap aktif, dan onChanged mengubah themeMode.value.
Theme.of(context)	Mengambil ThemeData yang sedang aktif, misalnya textTheme.titleMedium untuk gaya teks, sehingga gaya otomatis menyesuaikan tema.
Wrap	Menyusun lingkaran pilihan warna berjajar ke samping dan otomatis turun baris bila ruang tidak cukup. spacing mengatur jarak antar lingkaran.
GestureDetector	Mendeteksi ketukan pada lingkaran warna. Pada onTap, nilai seedColor.value diganti dengan warna yang diketuk.
CircleAvatar	Menampilkan lingkaran berwarna. Ikon centang ditampilkan pada warna yang sedang terpilih.

Bagian B. Named Routes, Argumen, dan Hero

Bagian ini menambahkan model data Item, halaman daftar (BerandaTab), dan halaman DetailPage. Perpindahan halaman memakai named route /detail yang membawa objek Item sebagai argumen. Ikon pada daftar dan pada halaman detail dibungkus Hero dengan tag yang sama sehingga ikon tampak terbang dan membesar saat berpindah halaman.

```
class Item {
  final String nama;
  final IconData ikon;
  final Color warna;
  const Item(this.nama, this.ikon, this.warna);
}

// BerandaTab: setiap baris daftar
ListTile(
  leading: Hero(
    tag: 'ikon-${item.nama}',
    child: CircleAvatar(backgroundColor: item.warna,
      child: Icon(item.ikon, color: Colors.white)),
  ),
  title: Text(item.nama),
  onTap: () => Navigator.pushNamed(context, '/detail', arguments: item),
);

// MyApp
initialRoute: '/',
routes: { '/': (_) => const ShellPage() },
onGenerateRoute: (settings) {
  if (settings.name == '/detail') {
    final item = settings.arguments as Item;
    return MaterialPageRoute(
      builder: (_) => DetailPage(item: item), settings: settings);
  }
  return null;
},
```

Bagian Kode	Penjelasan
class Item	Model data sederhana berisi nama, ikon, dan warna. Konstruktor const membuat objek konstan sehingga hemat memori.
daftarItem	Daftar konstan berisi empat item (Flutter, Musik, Kamera, Peta) yang menjadi sumber data daftar.
ListView.builder	Membuat daftar secara lazy, yaitu hanya item yang terlihat yang dibangun. itemCount menentukan jumlah item dan itemBuilder membangun tiap baris.
ListTile	Baris standar dengan leading (ikon di kiri), title, trailing (panah di kanan), dan onTap untuk aksi ketukan.
Hero	Menandai elemen yang berpindah antar halaman. Dua Hero harus memiliki tag yang sama dan unik pada halaman yang sama.
Navigator.pushNamed()	Membuka halaman berdasarkan nama rute dan membawa arguments berupa objek Item yang diketuk.
DetailPage	Halaman detail yang menerima Item lewat konstruktor. Ikonnya memakai Hero dengan radius: 64 sehingga tampak membesar, dan teksnya memakai Theme.of(context).textTheme.

Bagian Kode	Penjelasan
initialRoute dan routes	initialRoute menentukan rute awal, sedangkan routes mendaftarkan rute tanpa argumen, yaitu /.
onGenerateRoute	Dipanggil untuk rute yang tidak ada di routes. Fungsi ini memeriksa settings.name, mengambil settings.arguments dan mengubahnya menjadi Item, lalu mengembalikan MaterialPageRoute. Bila nama tidak dikenal, dikembalikan null. Pengecekan tipe di satu tempat membuatnya lebih aman daripada membaca argumen langsung di halaman tujuan.
MaterialPageRoute	Membuat transisi halaman standar Material menuju widget yang dibangun oleh builder.

Bagian C. Animasi

AnimasiTab memuat tiga demo animasi: animasi implisit dengan AnimatedContainer, AnimatedSwitcher untuk pergantian angka, dan animasi eksplisit dengan AnimationController untuk memutar ikon roda gigi. Karena AnimationController membutuhkan vsync, kelas state memakai SingleTickerProviderStateMixin.

```
late final AnimationController _putar;

@override
void initState() {
  super.initState();
  _putar = AnimationController(
    vsync: this,
    duration: const Duration(seconds: 2),
  )..repeat();
}

@override
void dispose() {
  _putar.dispose();
  super.dispose();
}

AnimatedContainer(
  duration: const Duration(milliseconds: 500),
  curve: Curves.easeInOut,
  width: _besar ? 200 : 100,
  height: 100,
  decoration: BoxDecoration(
    color: _besar ? warna.primary : warna.tertiary,
    borderRadius: BorderRadius.circular(_besar ? 50 : 8)),
)

AnimatedSwitcher(
  duration: const Duration(milliseconds: 300),
  transitionBuilder: (child, anim) =>
      ScaleTransition(scale: anim, child: child),
  child: Text('$_hitung', key: ValueKey(_hitung)),
)

RotationTransition(turns: _putar, child: Icon(Icons.settings))
```

Bagian Kode	Penjelasan
SingleTickerProviderStateMixin	Menyediakan Ticker yang mengatur denyut frame untuk animasi. Tanpa mixin ini, vsync: this menimbulkan error TickerProvider.

Bagian Kode	Penjelasan
AnimationController	Pengendali animasi eksplisit. duration menentukan lama satu putaran (2 detik). Method repeat() mengulang tanpa henti, stop() menghentikan, dan isAnimating memberi tahu apakah sedang berjalan. Tanda .. (cascade) memanggil repeat() pada objek yang baru dibuat.
initState()	Dipanggil sekali saat state dibuat. Di sini controller diinisialisasi.
dispose()	Dipanggil saat widget dihapus. _putar.dispose() wajib dipanggil agar ticker dilepas dan tidak terjadi kebocoran memori.
AnimatedContainer	Animasi implisit: bila width, color, atau borderRadius berubah lewat setState, perubahannya dianimasikan selama duration dengan kurva Curves.easeInOut.
colorScheme (primary, tertiary)	Warna diambil dari tema aktif, bukan ditulis manual, sehingga ikut berganti saat tema berubah.
AnimatedSwitcher	Menganimasikan pergantian child. transitionBuilder menentukan efek, di sini ScaleTransition sehingga angka tampak membesar.
ValueKey	Memberi identitas berbeda pada tiap angka sehingga Flutter menganggapnya widget baru dan menjalankan animasi. Tanpa key yang berbeda, animasi tidak terjadi.
RotationTransition	Memutar anaknya mengikuti nilai turns. Nilai 1 berarti satu putaran penuh (360 derajat).
setState()	Memberi tahu Flutter agar membangun ulang tampilan, misalnya untuk mengganti label tombol Berhenti/Putar.

Bagian D. Navigasi Tab Bawah

ShellPage menyatukan seluruh bagian menjadi aplikasi dengan tiga tab di bawah: Beranda, Animasi, dan Pengaturan. Rute / pada MyApp diubah agar menampilkan ShellPage.

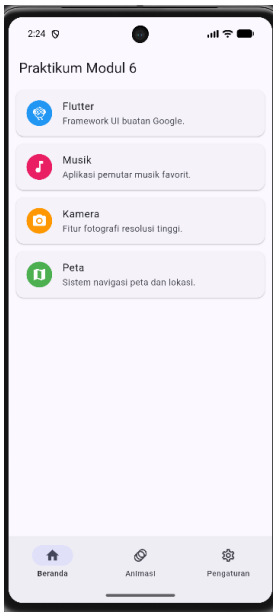
```
class  ShellPageState extends State<ShellPage> {
  int _index = 0;
  static const _halaman = [BerandaTab(), AnimasiTab(), PengaturanTab()];
  static const _judul = ['Beranda', 'Animasi', 'Pengaturan'];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text( judul[ index])),
      body: IndexedStack(index: _index, children: _halaman),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _index,
        onDestinationSelected: (i) => setState(() => _index = i),
        destinations: const [ NavigationDestination(...), ... ],
      ),
    );
  }
}
```

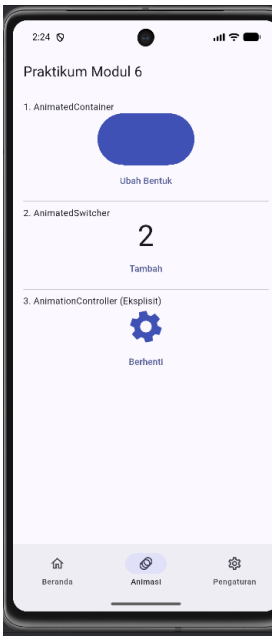
Bagian Kode	Penjelasan
ShellPage	Halaman kerangka berbentuk StatefulWidget karena harus menyimpan tab yang sedang aktif.

Bagian Kode	Penjelasan
_index	Variabel state yang menyimpan nomor tab aktif (0 = Beranda, 1 = Animasi, 2 = Pengaturan).
IndexedStack	Menampilkan satu anak sesuai index, sementara anak lainnya tetap hidup di belakang sehingga state tiap tab terjaga saat berpindah tab.
NavigationBar	Bilah navigasi bawah Material 3. selectedIndex menandai tab aktif dan onDestinationSelected dipanggil saat tab diketuk.
NavigationDestination	Satu item pada bilah navigasi, terdiri dari icon, selectedIcon (ikon saat terpilih), dan label.
AppBar(title: _judul[_index])	Judul AppBar mengikuti tab yang sedang aktif.
routes: {'/': ...}	Rute awal diarahkan ke ShellPage sehingga aplikasi langsung membuka tampilan bertab.

Hasil Praktikum



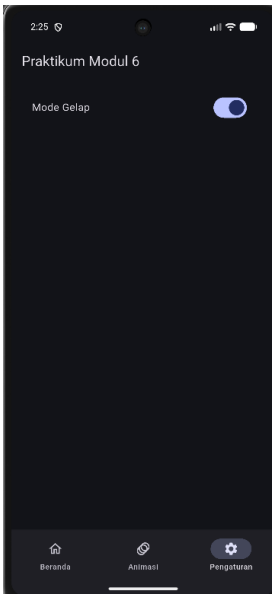
Gambar 1. Tab Beranda



Gambar 2. Tab Animasi



Gambar 3. Pengaturan, mode terang



Gambar 4. Pengaturan, mode gelap

Gambar 1 menampilkan tab Beranda berisi empat item (Flutter, Musik, Kamera, Peta) beserta navigasi bawah dengan tiga tab. Mengetuk salah satu item membuka halaman detail dengan transisi Hero. Gambar 2 menampilkan tab Animasi: kotak AnimatedContainer berubah bentuk secara halus, angka pada AnimatedSwitcher bertambah dengan efek membesar, dan ikon roda gigi berputar terus hingga tombol Berhenti ditekan. Gambar 3 dan Gambar 4 menampilkan tab Pengaturan: saklar Mode Gelap mengubah seluruh tampilan, termasuk AppBar dan navigasi bawah, dari terang menjadi gelap tanpa restart aplikasi.

5. Latihan Mandiri

Latihan mandiri dikerjakan sebagai pengembangan aplikasi praktikum: (1) menyimpan mode gelap dan warna tema dengan shared_preferences, (2) menambahkan Drawer pada ShellPage, (3) mengganti transisi halaman detail dengan PageRouteBuilder dan FadeTransition, (4) menambahkan onUnknownRoute, dan (5) menambahkan demo AnimatedOpacity pada AnimasiTab.

1. Menyimpan pengaturan tema dengan shared_preferences

Mode gelap dan warna tema disimpan ke penyimpanan lokal agar tetap terpakai setelah aplikasi ditutup, lalu dimuat saat aplikasi dimulai.

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final prefs = await SharedPreferences.getInstance();
  themeMode.value = (prefs.getBool('gelap') ?? false)
      ? ThemeMode.dark : ThemeMode.light;
  seedColor.value = Color(prefs.getInt('warna') ?? Colors.indigo.value);
  themeMode.addListener(() =>
      prefs.setBool('gelap', themeMode.value == ThemeMode.dark));
  seedColor.addListener(() =>
      prefs.setInt('warna', seedColor.value.value));
  runApp(const MyApp());
}
```

Bagian Kode	Penjelasan
WidgetsFlutterBinding.ensureInitialized()	Memastikan binding Flutter siap sebelum memanggil kode asinkron di main().
SharedPreferences.getInstance()	Membuka penyimpanan lokal. getBool dan getInt membaca nilai tersimpan, dan operator ?? memberi nilai bawaan bila data belum ada.
addListener	Dipasang pada notifier agar setiap kali pengguna mengubah tema, nilai barunya disimpan dengan setBool dan setInt.

2. Drawer pada ShellPage

Drawer ditambahkan pada Scaffold sebagai menu samping yang berisi daftar tujuan. Setiap ListTile mengubah _index melalui setState lalu menutup drawer dengan Navigator.pop(context). Ikon tiga garis (hamburger) pada AppBar muncul otomatis dan dapat dilihat pada Gambar 5 dan Gambar 6.

3. Transisi halaman dengan PageRouteBuilder

Transisi halaman detail diganti menjadi efek memudar. Di dalam onGenerateRoute, MaterialPageRoute digantikan PageRouteBuilder yang memakai FadeTransition.

```
return PageRouteBuilder(
  settings: settings,
  pageBuilder: ( ,   ,    ) => DetailPage(item: item),
  transitionsBuilder: (_, anim, __, child) =>
      FadeTransition(opacity: anim, child: child),
);
```

pageBuilder membangun halaman tujuan, sedangkan transitionsBuilder menentukan animasi perpindahan. Parameter anim bernilai 0 sampai 1 sehingga halaman muncul dari transparan menjadi penuh.

4. onUnknownRoute

onUnknownRoute dipanggil ketika nama rute tidak ditemukan di routes maupun onGenerateRoute. Fungsi ini menampilkan halaman "404 - Halaman tidak ditemukan" sehingga aplikasi tidak error.

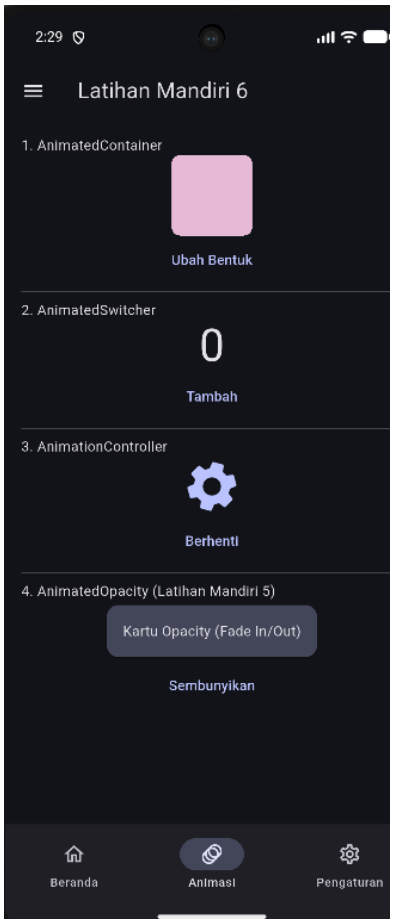
```
onUnknownRoute: (settings) => MaterialPageRoute(
  builder: (_) => Scaffold(
    appBar: AppBar(title: const Text('404')),
    body: const Center(child: Text('404 - Halaman tidak ditemukan')),
  ),
),
```

5. AnimatedOpacity pada AnimasiTab

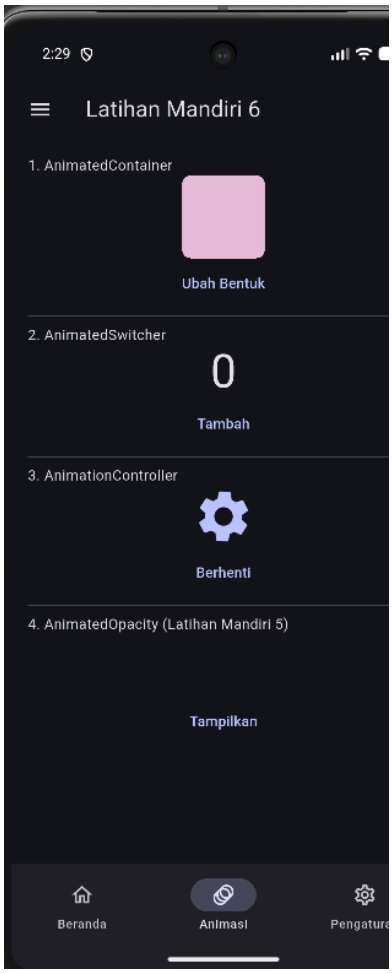
Demo keempat ditambahkan pada AnimasiTab berupa kartu yang memudar dan muncul kembali. Variabel boolean _tampil menentukan nilai opacity (1 atau 0). Perubahan nilai dianimasikan otomatis selama duration, dan label tombol berganti antara Sembunyikan dan Tampilkan.

```
AnimatedOpacity(
  opacity: _tampil ? 1.0 : 0.0,
  duration: const Duration(milliseconds: 500),
  child: Card(child: Padding(
    padding: const EdgeInsets.all(16),
    child: Text('Kartu Opacity (Fade In/Out)'))),
),
TextButton(
  onPressed: () => setState(() => _tampil = !_tampil),
  child: Text(_tampil ? 'Sembunyikan' : 'Tampilkan'),
),
```

Gambar 5 memperlihatkan kartu dalam keadaan tampil, sedangkan Gambar 6 memperlihatkan kartu yang telah memudar dan tombol berubah menjadi Tampilkan.



Gambar 5. Kartu opacity tampil



Gambar 6. Kartu opacity tersembunyi

6. Tugas: Aplikasi Katalog Wisata

Tugas berupa aplikasi Katalog Interaktif bertema destinasi wisata. Katalog berisi empat destinasi, yaitu Candi Borobudur, Pantai Kuta, Gunung Bromo, dan Raja Ampat. Ketentuan tugas dan implementasinya dirangkum pada tabel berikut.

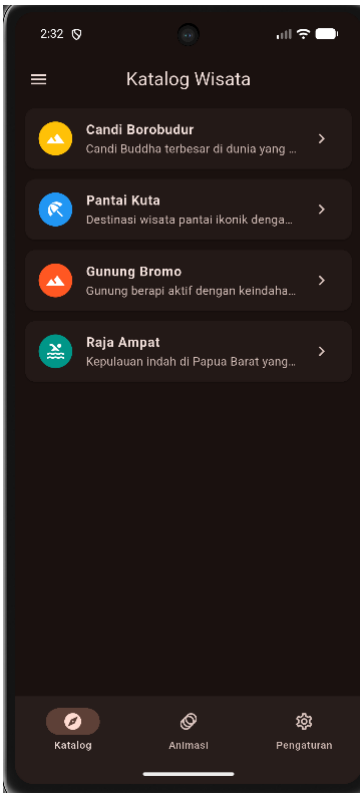
Ketentuan	Implementasi
Minimal 3 tab dengan NavigationBar	Tiga tab: Katalog, Animasi, dan Pengaturan, dikelola oleh NavigationBar dan IndexedStack. Menu samping (Drawer) juga tersedia.
Named routes dengan argumen dan Hero	Mengetuk destinasi membuka halaman detail melalui Navigator.pushNamed dengan argumen objek destinasi yang dibaca di onGenerateRoute. Ikon destinasi dibungkus Hero.
Minimal dua jenis animasi	Implisit: AnimatedContainer dan AnimatedOpacity. Eksplisit: AnimationController dengan RotationTransition pada ikon yang berputar.
Tema terang/gelap dan minimal 3 warna, tersimpan	Saklar Mode Gelap dan empat pilihan warna (indigo, teal, deepOrange, pink). Pilihan disimpan dengan shared_preferences dan dimuat saat aplikasi dibuka.

Ketentuan	Implementasi
Warna dan gaya teks dari Theme.of(context)	Warna latar, teks, dan ikon memakai Theme.of(context).colorScheme dan textTheme, sehingga seluruh aplikasi mengikuti tema yang dipilih.

Tab Katalog menampilkan daftar destinasi memakai ListView.builder. Setiap baris berupa ListTile yang memuat ikon berwarna, nama destinasi, deskripsi singkat yang dipotong dengan maxLines dan overflow: TextOverflow.ellipsis, serta panah di kanan sebagai petunjuk bahwa baris dapat diketuk. Warna tampilan berasal dari tema (Gambar 7).

Tab Demo Animasi memuat tiga animasi: AnimatedContainer yang mengubah ukuran dan warna (implisit), AnimatedOpacity yang membuat kartu memudar dan muncul (implisit), serta AnimationController yang memutar ikon tanpa henti dan dapat dihentikan dengan tombol Berhenti (eksplisit). Tampilan ditunjukkan pada Gambar 8.

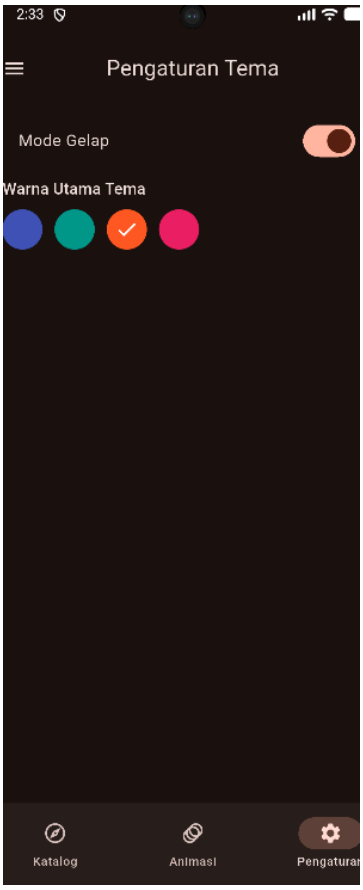
Tab Pengaturan Tema memuat saklar Mode Gelap dan empat lingkaran pilihan warna. Warna yang sedang aktif ditandai dengan ikon centang. Pada Gambar 9, mode gelap aktif dan warna deepOrange terpilih, sehingga seluruh aplikasi, termasuk AppBar dan navigasi bawah, berubah menjadi nuansa oranye gelap. Fungsi-fungsi yang digunakan sama dengan Bagian A pada Hasil Praktikum, ditambah penyimpanan ke shared_preferences seperti pada Latihan Mandiri nomor 1.



Gambar 7. Tab Katalog



Gambar 8. Tab Animasi



Gambar 9. Tab Pengaturan

7. Jawaban Pertanyaan Refleksi

1. Apa perbedaan animasi implisit dan eksplisit? Kapan memilih masing-masing?

Pada animasi implisit, pengembang cukup mengubah nilai properti melalui setState dan Flutter menganimasikan perubahannya secara otomatis (contoh AnimatedContainer). Pada animasi eksplisit, pengembang membuat dan mengendalikan AnimationController sendiri. Animasi implisit dipilih untuk perubahan sederhana dari satu nilai ke nilai lain, sedangkan animasi eksplisit dipilih bila dibutuhkan kontrol penuh seperti mengulang, menghentikan, membalik, atau menyinkronkan beberapa animasi.

2. Mengapa dua Hero harus memiliki tag yang sama, dan apa yang terjadi bila tag-nya kembali muncul dua kali di halaman yang sama?

Flutter mencocokkan Hero pada halaman asal dan halaman tujuan berdasarkan tag. Bila tag sama, Flutter tahu bahwa kedua widget adalah elemen yang sama dan menganimasikan perpindahan di antaranya. Jika tag yang sama muncul dua kali pada satu halaman, Flutter tidak dapat menentukan pasangan yang benar sehingga terjadi error tag duplikat atau animasi tidak berjalan. Karena itu tag dibuat unik per item, misalnya dengan menyertakan nama item.

3. Apa keuntungan memakai Theme.of(context) dibanding menulis warna langsung pada widget?

Warna dan gaya terpusat di satu tempat sehingga perubahan tema cukup dilakukan sekali dan seluruh widget ikut berubah. Warna dari tema juga otomatis menyesuaikan mode

terang/gelap sehingga teks tetap terbaca. Selain itu, tampilan menjadi konsisten dan kode lebih rapi karena tidak ada nilai warna yang tersebar.

4. Apa perbedaan berpindah tab dengan IndexedStack dan membuka halaman dengan Navigator?

IndexedStack menampung semua tab sekaligus dan hanya menampilkan satu, sehingga state tiap tab tetap terjaga (posisi scroll dan animasi tidak hilang). Perpindahan tab bersifat sejajar dan tidak menambah riwayat halaman. Navigator menumpuk halaman baru di atas halaman lama, ada tombol kembali, dan halaman yang ditutup dihancurkan beserta state-nya. IndexedStack cocok untuk menu utama sedangkan Navigator cocok untuk halaman lanjutan seperti detail.

5. Mengapa AnimationController wajib di-dispose?

AnimationController memakai Ticker yang terus berdetak setiap frame. Jika widget dihapus tanpa memanggil dispose(), ticker tetap berjalan sehingga memboroskan sumber daya, menyebabkan kebocoran memori, dan dapat memicu error karena setState dipanggil pada widget yang sudah tidak ada.

8. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa:

- Tema Material 3 dapat dibangun dari satu warna dasar melalui colorSchemeSeed. Dengan ValueNotifier dan ListenableBuilder, tema terang/gelap dan warna aplikasi dapat diganti langsung tanpa restart.
- Memakai Theme.of(context) membuat seluruh tampilan konsisten dan otomatis menyesuaikan tema.
- Named routes bersama onGenerateRoute membuat navigasi lebih terpusat dan aman untuk mengirim argumen, sedangkan Hero memberi transisi visual yang halus antar halaman.
- NavigationBar dan IndexedStack memudahkan pembuatan navigasi tab bawah dengan state yang tetap terjaga.
- Animasi implisit cocok untuk perubahan sederhana, sedangkan animasi eksplisit dengan AnimationController memberi kontrol penuh dan wajib di-dispose.
- Pengaturan tema dapat disimpan dengan shared_preferences sehingga tetap ada setelah aplikasi ditutup. Aplikasi Katalog Wisata pada tugas menjadi bahan awal untuk mini proyek di Pertemuan 7.